

Taller Postman

Angel David Ruiz Barbosa

aruizba@unal.edu.co

1.)

¿Qué API elegiste y por qué? → SpaceX porque no requiere token, tiene endpoints REST bien diseñados con recursos relacionados (lanzamientos, cohetes, misiones), y soporta POST de consulta avanzada.

¿Qué datos devuelve? → Objetos con campos como `name`, `date_utc`, `success`, `links`, `rocket` (ID), `payloads` (array de IDs), entre muchos más.

¿Usa token o no? → No usa token. Es una API completamente pública.

¿Qué código de estado recibiste? → `200 OK` en todos los GETs exitosos, también `200` en el POST de query.

¿Qué aprendiste diferente a JSONPlaceholder? → JSONPlaceholder es ficticio y simplificado; SpaceX devuelve datos reales con estructuras más complejas, IDs UUID, fechas ISO 8601, relaciones entre recursos (un lanzamiento referencia cohetes y payloads por ID), y soporta queries con filtros y paginación mediante POST.

Análisis y Profundización de la Selección y Uso de la API de SpaceX

La elección de la API de SpaceX para el ejercicio de integración se fundamentó en varios criterios clave que la distinguen de otras opciones y la hacen ideal para demostrar capacidades de cliente REST robustas.

1. Elección de la API y Justificación Detallada:

- **API Seleccionada:** SpaceX Public REST API (la API no oficial pero ampliamente utilizada que consolida los datos de sus lanzamientos).
- **Justificación Principal:** La principal ventaja radica en su naturaleza de acceso abierto y totalmente pública, ya que **no requiere la gestión de tokens de autenticación (Auth Tokens)** ni claves API para la mayoría de sus endpoints de lectura. Esto simplifica drásticamente la configuración inicial y el manejo de credenciales.
- **Arquitectura REST Madura:** La API ofrece un conjunto de endpoints RESTful bien diseñados y lógicos. Los recursos como `/launches`, `/rockets`, y `/cores` están claramente definidos, permitiendo la navegación y recuperación de datos de manera intuitiva.
- **Relaciones Explícitas entre Recursos:** A diferencia de APIs muy sencillas, la de SpaceX implementa relaciones claras. Por ejemplo, la respuesta de un lanzamiento (`/launches`) incluye referencias (IDs) a los cohetes (`rocket`), las etapas centrales (`cores`) y las cargas útiles (`payloads`) involucradas. Esta interconexión es crucial para construir aplicaciones con lógica de negocio compleja.
- **Capacidad de Consulta Avanzada (POST Query):** Una característica distintiva y

avanzada es el soporte para la consulta de recursos mediante peticiones **POST** al endpoint `/query`. Esta funcionalidad permite a los desarrolladores enviar cuerpos de solicitud complejos con filtros anidados, opciones de paginación (`limit`, `offset`) y criterios de ordenación (`sort`), lo cual es significativamente más potente que las simples consultas GET con *query parameters*.

2. Estructura y Naturaleza de los Datos Devueltos:

Los datos retornados por la API de SpaceX son objetos JSON ricos en detalles, reflejando la complejidad de las operaciones espaciales.

- **Campos Clave en los Objetos de Lanzamiento:** Un objeto de lanzamiento (`launch`) típico incluye:
 - `name`: Nombre de la misión.
 - `date_utc`: Fecha y hora del lanzamiento en formato estándar ISO 8601 (por ejemplo, `2020-05-30T19:22:00.000Z`).
 - `success`: Booleano que indica el éxito de la misión.
 - `links`: Objeto anidado que contiene URLs a recursos multimedia (videos de YouTube, imágenes de Flickr, enlaces a artículos de prensa).
 - `rocket`: Una cadena (string) que contiene el **UUID** del cohete utilizado (estableciendo la relación).
 - `payloads`: Un **array de UUIDs** que identifica todas las cargas útiles transportadas en el lanzamiento.
- **Tipos de Datos Reales:** Los identificadores utilizados son UUIDs (Universally Unique Identifiers), y las marcas de tiempo se adhieren al estándar ISO 8601, lo que es característico de APIs de datos reales y de alta calidad, contrastando con IDs secuenciales simples.

3. Autenticación y Acceso:

- **Uso de Tokens:** No utiliza token ni ningún tipo de autenticación OAuth 2.0 o API Key para las operaciones de lectura de datos.
- **Conclusión:** Es una **API completamente pública** diseñada para la distribución masiva de datos de sus misiones.

4. Códigos de Estado HTTP Recibidos:

- **Peticiones GET Exitosas:** El código de respuesta estándar para la recuperación de un recurso (individual o lista) es **200 OK**.
- **Peticiones POST de Consulta Avanzada:** A pesar de ser un método POST, cuando se utiliza para la funcionalidad de *querying* (que es esencialmente una búsqueda filtrada), el código de estado de éxito esperado y recibido es también **200 OK**. Esto confirma que la solicitud fue procesada y la respuesta contiene los datos solicitados.

5. Diferencias Cruciales con JSONPlaceholder:

La transición de JSONPlaceholder a la API de SpaceX representa un salto de un entorno simulado a un entorno de datos real, ofreciendo lecciones valiosas:

Característica	JSONPlaceholder (Ficticio y Simplificado)	SpaceX API (Datos Reales y Complejos)
Naturaleza de los Datos	Ficticios, simples (posts, comentarios, usuarios).	Reales, detallados, y específicos de un dominio (cohetes, lanzamientos, cápsulas).
Estructura de ID	IDs numéricos secuenciales (1, 2, 3...).	UUIDs (Identificadores Universalmente Únicos) , de longitud y complejidad mayor.
Manejo de Fechas	A menudo, solo <i>timestamps</i> o cadenas de texto simples.	Formato estándar ISO 8601 (YYYY-MM-DDTHH:mm:ss.sssZ).
Relaciones entre Recursos	Generalmente mediante <i>userId</i> o <i>postId</i> simple.	Relaciones explícitas mediante IDs de recursos relacionados (<i>rocket</i> ID, <i>payloads</i> array de IDs).
Funcionalidad de Consulta	Limitada a parámetros GET simples (e.g., <i>?userId=1</i>).	Soporte avanzado de POST para querying , permitiendo filtros complejos, rangos, y opciones de paginación (<i>limit</i> , <i>offset</i>) en el cuerpo de la solicitud.

En resumen, trabajar con la API de SpaceX expuso la necesidad de manejar estructuras de datos más ricas, entender las relaciones entre múltiples recursos mediante UUIDs, y aprovechar métodos HTTP avanzados (como POST para consultas filtradas) más allá de las operaciones CRUD básicas.

2.)

¿Qué diferencia encontraste vs REST? → En REST necesitas conocer múltiples endpoints y recibes todos los campos del recurso (over-fetching), o a veces demasiado pocos (under-fetching). En GraphQL hay un solo endpoint, tú defines exactamente qué campos quieres, y puedes traversal relaciones en una sola petición.

¿Cuántos requests REST necesitarías para reemplazar tu query más compleja? → La Query 4 (continente → países → idiomas) requeriría al menos 3 requests en REST: uno para el continente, otro para listar sus países, y otro por cada país para obtener sus idiomas. Con 12 países en Sudamérica serían ~14 requests. En GraphQL es 1.

¿En qué proyecto real usarías GraphQL? → En el frontend de badges de la UIFCE sería ideal: cuando una vista necesita datos del usuario, sus badges y las categorías de esos

badges, actualmente requeriría 3 llamadas REST; con GraphQL sería una sola query con los campos exactos que necesita cada componente Angular.

Comparativa y Aplicaciones de GraphQL vs. REST1. Diferencias Fundamentales entre GraphQL y REST

La principal disparidad entre ambas arquitecturas reside en cómo se estructura la comunicación y el acceso a los datos.

Característica	REST	GraphQL
Puntos de Acceso (Endpoints)	Múltiples y específicos para cada recurso (ej: <code>/usuarios</code> , <code>/productos/123</code> , <code>/usuarios/1/pedidos</code>).	Un único endpoint (comúnmente <code>/graphql</code>) para todas las peticiones.
Definición de Datos	El servidor define la estructura de los datos que se entregan para cada endpoint.	El cliente define exactamente qué campos y relaciones de campos necesita en su <i>query</i> .
Problemas Comunes	Over-fetching (recibir más campos de los necesarios) o Under-fetching (recibir muy pocos, obligando a hacer más peticiones).	Se mitigan ambos problemas; solo se reciben los datos solicitados.
Manejo de Relaciones	Requiere múltiples peticiones para "traversar" (seguir) relaciones entre recursos (ej: un <code>GET /usuario/1</code> seguido de un <code>GET /usuario/1/direcciones</code>).	Permite traversar relaciones de manera anidada en una sola petición.

Conclusión: Mientras que REST se basa en recursos y verbos HTTP, GraphQL se centra en las necesidades específicas del dato que tiene el cliente, otorgando un control sin precedentes sobre la información transferida.**2. Eficiencia en la Reducción de Peticiones**

La capacidad de GraphQL para consolidar datos se traduce directamente en una drástica reducción de peticiones al servidor, optimizando la latencia y el uso de ancho de banda, especialmente en redes móviles o con alta latencia.

Caso de Uso: Obtener Continente, Países e Idiomas (Query Anidada)

Imaginemos una aplicación que necesita mostrar una lista de países dentro de un continente específico y, para cada país, su lista de idiomas oficiales.

- **En GraphQL:**
 - **Peticiones necesarias: 1**
 - Una única *query* al *endpoint* `/graphql` es suficiente. La *query* podría lucir algo como: `query { continente(nombre: "Sudamérica") { nombre paises { nombre idiomas { nombre } } } }`.
- **En REST (con 12 países en Sudamérica):**
 - **Peticiones necesarias (Estimación): ~14**
 - **Petición 1:** `GET /continentes/sudamerica` (Para obtener la lista inicial de países o solo el continente).
 - **Petición 2:** `GET /continentes/sudamerica/paises` (Para obtener la lista de 12 países).
 - **Peticiones 3 a 14:** `GET /paises/{id_pais}/idiomas` (Una petición individual por cada uno de los 12 países para obtener sus respectivos idiomas).

La diferencia de **14 requests** a **1 request** ilustra el potencial de optimización de GraphQL, haciendo que la aplicación sea más rápida y reactiva.

3. Escenario de Aplicación Real: El Frontend de Badges de la UIFCE

GraphQL se presenta como una solución ideal para frontends complejos que consumen datos de múltiples fuentes o dominios dentro de un mismo sistema.

Contexto del Proyecto (UIFCE Badges): Se requiere construir una vista que muestre información del usuario junto con las insignias (badges) que ha obtenido y las categorías a las que pertenecen dichas insignias.

Situación Actual (con REST):

Para cargar una sola vista, el frontend de Angular (o cualquier otro) probablemente necesitaría ejecutar **tres (3) llamadas REST secuenciales o paralelas**:

1. `GET /api/v1/usuarios/{id}`: Para obtener los datos básicos del usuario (nombre, foto, etc.).
2. `GET /api/v1/usuarios/{id}/badges`: Para obtener la lista de insignias del usuario (nombre de la insignia, fecha de obtención).
3. `GET /api/v1/badges/{id}/categoria`: Para obtener el detalle o la categoría de cada insignia. *Si el sistema tiene 5 insignias, podrían ser hasta 5 llamadas adicionales solo para las categorías, agravando el under-fetching.*

Solución Propuesta (con GraphQL):

El frontend puede obtener toda la información necesaria en **una sola query**:

- **Una Sola Petición:** Se envía una única *query* que solicita al mismo tiempo los datos del usuario, los badges asociados, y los detalles de sus categorías.
- **Campos Exactos:** Cada componente de Angular recibe **únicamente** los campos que necesita. Por ejemplo:

- El componente de Perfil solo pide el **nombre** y la **foto** del usuario.
- El componente de Badges solo pide el **título** y la **imagen** del badge, y el **nombre** de la **categoría** anidada.

Esta consolidación reduce la carga del servidor, simplifica la lógica del cliente (ya no tiene que coordinar múltiples respuestas), y mejora significativamente la experiencia del usuario final debido a tiempos de carga reducidos.